Proving Data Structures Correct: Balanced Search Trees

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Search Time Depends on the Height

Week 4: `isin` follows one path from the root

©Mohammad Abdulaziz

Search Time Depends on the Height

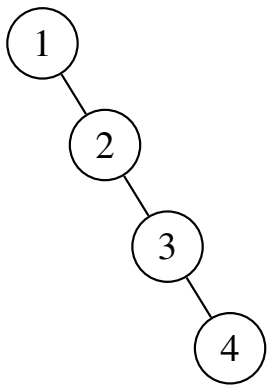
Week 4: `isin` follows one path from the root
$\Rightarrow$ search takes time $O(h(t))$, where $h(t)$ is the height of t

©Mohammad Abdulaziz

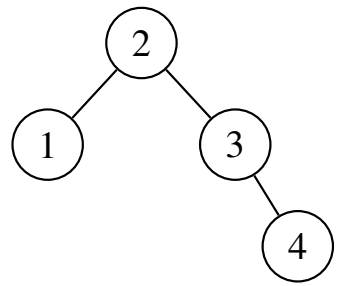
Search Time Depends on the Height

Week 4: `isin` follows one path from the root
$\Rightarrow$ search takes time $O(h(t))$, where $h(t)$ is the height of t

The height depends on the order of insertion:



`ins` 1, 2, 3, 4: height 4



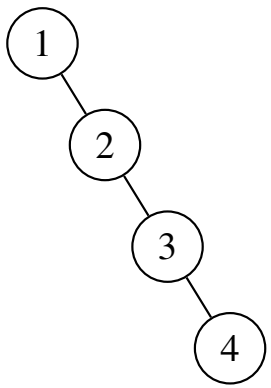
`ins` 2, 1, 3, 4: height 3

©Mohammad Abdulaziz

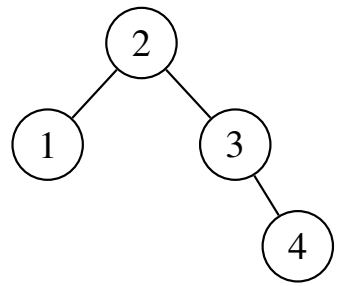
Search Time Depends on the Height

Week 4: `isin` follows one path from the root
$\Rightarrow$ search takes time $O(h(t))$, where $h(t)$ is the height of t

The height depends on the order of insertion:



`ins` 1, 2, 3, 4: height 4



`ins` 2, 1, 3, 4: height 3

Worst case: $h(t) = |t|$

Best case: $h(t) \approx \log_2 |t|$

Based on Tobias Nipkow's *Functional Data Structures* slides.

©Mohammad Abdulaziz

Balanced Search Trees

Balanced search trees guarantee $h(t) = O(\log |t|)$

©Mohammad Abdulaziz

Balanced Search Trees

Balanced search trees guarantee $h(t) = O(\log |t|)$

Two kinds of invariants:

- the ordering invariant, like `bst` in Week 4

Balanced Search Trees

Balanced search trees guarantee $h(t) = O(\log |t|)$

Two kinds of invariants:

- the ordering invariant, like `bst` in Week 4
- a structural invariant = a *balance criterion*

©Mohammad Abdulaziz

Balanced Search Trees

Balanced search trees guarantee $h(t) = O(\log |t|)$

Two kinds of invariants:

- the ordering invariant, like `bst` in Week 4
- a structural invariant = a *balance criterion*

`insert` (and `delete`) must preserve both.

©Mohammad Abdulaziz

Balanced Search Trees

Balanced search trees guarantee $h(t) = O(\log |t|)$

Two kinds of invariants:

- the ordering invariant, like `bst` in Week 4
- a structural invariant = a *balance criterion*

`insert` (and `delete`) must preserve both.

Examples: AVL trees, red-black trees, 2-3 trees, B-trees, ...

©Mohammad Abdulaziz